DigitalTwin.ai

Spatio-Temporal Graph Intelligence for Vehicle Assembly Lines

> **Accenture Innovation Challenge 2026 — Round 2 Prototype Submission**
> **Author:** Piyush Maji

🚀 Problem Statement

Modern vehicle assembly lines are tightly coupled networks where small disruptions cascade rapidly. A brief stoppage upstream ripples downstream into bottlenecks and buffer overflows. Defects go undetected until end-of-line QC, causing expensive rework. Current plant monitoring is **static and retrospective** — dashboards and siloed PLC telemetry only alert after thresholds are breached, and sparse sensor coverage on legacy equipment leaves blind spots.

Assembly lines mix **legacy and modern equipment** with highly uneven sensor coverage — some stations are richly instrumented, others rely entirely on manual checklists. Bottlenecks and defects often have **multi-causal, intermittent root causes** (equipment wear, operator variation, upstream part quality, environmental conditions) that are hard to isolate from data alone. A defect introduced early in the line may not surface until a much later inspection point, making **root-cause tracing after the fact especially difficult**.

There's no live, interconnected model that captures spatial and temporal dependencies across stations to predict problems before they cascade.

💡 Solution

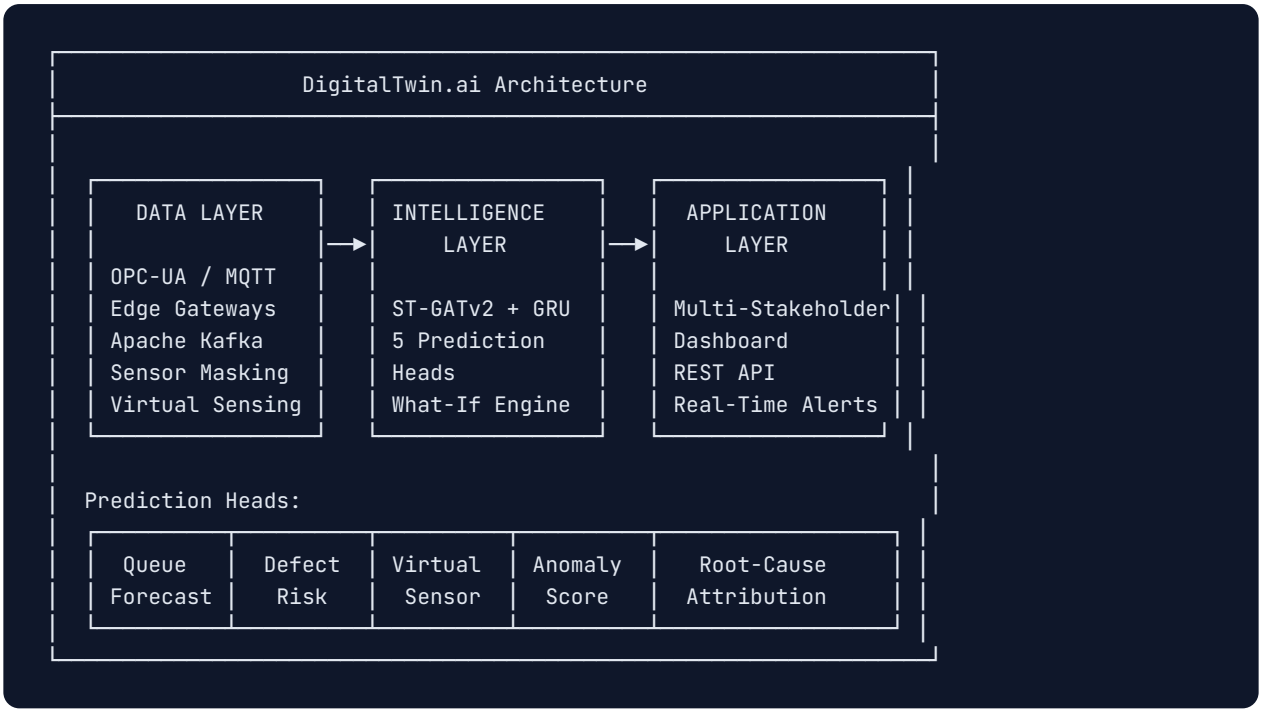
DigitalTwin.ai models the entire assembly line as a **dynamic directed graph** — stations as nodes, material/information flow as edges — powered by a **Spatio-Temporal Graph Neural Network (ST-GNN)**. The system provides **three distinct stakeholder views** from the same underlying model.

Core Capabilities

CAPABILITY	HOW IT WORKS	IMPACT
Predictive Propagation	GATv2 + GRU layers learn spatial and temporal dependencies, forecasting queue build-ups and classifying defect-propagation risk across downstream stations	Prevents cascading bottlenecks with 10-15 min advance warning

CAPABILITY	HOW IT WORKS	IMPACT
Virtual Sensing	Masked-node training lets the GNN infer state of under-instrumented legacy stations via neighbor aggregation, with per-node confidence scoring	Extends monitoring to legacy equipment without new hardware
What-If Simulation	Learned GNN dynamics overlaid on discrete-event line logic for testing interventions (maintenance, rerouting, takt changes) virtually	De-risks operational decisions before execution
Multi-Stakeholder UX	Floor supervisor (real-time alerts), Plant manager (shift planning), Leadership (ROI/validation) views from one model	Drives adoption across organizational levels
Cascade Detection	Graph-based anomaly propagation tracking with multi-hop downstream impact analysis	Identifies root causes, not just symptoms

🏗️ Architecture



Model Details (v2)

COMPONENT	SPECIFICATION
Graph Type	Dynamic directed graph (40 stations, 48 edges)
Zones	Body Construction (13) · Paint Shop (11) · Final Assembly (16)

COMPONENT	SPECIFICATION
Spatial Layer	GATv2Conv (4 attention heads)
Temporal Layer	GRU cell per node
Stacked Blocks	3 × Spatio-Temporal GAT blocks
Input Features	12 per station (queue, cycle time, defect rate, temp, vibration, torque deviation, utilization, throughput, humidity, power draw, maintenance hours, operator skill)
Hidden Dimension	128
Prediction Heads	Queue forecast (regression), Defect risk (3-class), Virtual sensor (reconstruction), Anomaly score (binary), Root-cause attribution (attention)
Instrumentation Mix	FULL: 60% · PARTIAL: 25% · MANUAL: 15%
Equipment Vintage	Modern: 60% · Legacy: 40%
Framework	PyTorch + PyTorch Geometric

Anomaly Scenarios (Built-In)

The simulator includes 5 realistic, multi-causal anomaly scenarios:

SCENARIO	CATEGORY	STATIONS	DESCRIPTION
Weld Gun Electrode Wear	Equipment Wear	Spot Weld Cell 3	Progressive electrode degradation, detected downstream at Geometry Check
Paint Booth Humidity	Environmental	Color Booth, Clear Coat	HVAC fault causes orange peel and fish-eye defects
Stamping Die Wear	Equipment Wear	Stamping Press B (legacy)	Dimensional drift in fender panels propagates through welding
Shift Change Operator Drop	Operator	Manual trim stations	B-shift inexperience causes higher defect rates and cycle times
Upstream Part Quality	Supply Chain	Adhesive Bonding	New supplier adhesive with lower viscosity affects bond strength

📁 Repository Structure

```
accentureInnovation/
├── app.py                  # Flask web server — interactive dashboard backend
├── assembly_line.py        # 40-station assembly line definition (3 zones)
├── simulator.py            # Production data simulator with anomaly injection
├── model.py                # DigitalTwinGNN — ST-GATv2 + GRU architecture
├── demo.py                 # Standalone demo script (terminal + visualizations)
├── requirements.txt        # Python dependencies
├── README.md               # This file
├── templates/
│   └── index.html          # Dashboard SPA template (multi-stakeholder tabs)
├── static/
│   ├── dashboard.js        # Dashboard frontend logic (graph, KPIs, charts)
│   └── style.css           # Dark-themed dashboard styling
└── demo_outputs/           # Generated after running demo.py
    ├── 01_assembly_graph_predictions.png
    ├── 02_temporal_dashboard.png
    ├── 03_virtual_sensing.png
    └── 04_validation_metrics.png
```

⚡ Quick Start

Prerequisites

- Python 3.9+
- pip

Installation

```
# Clone the repository
git clone https://github.com/Piyush314159/DigitalTwinAI.git
cd DigitalTwinAI

# Install dependencies
pip install -r requirements.txt
```

> **Note on PyTorch Geometric:** If `torch-geometric` installation fails, follow the official installation guide for your specific PyTorch + CUDA version.

Run the Interactive Dashboard

```
python app.py
```

Open **http://localhost:5001** in your browser. The dashboard provides:
- **Floor Supervisor** tab — real-time assembly graph, alerts, station matrix, virtual sensing
- **Plant Manager** tab — shift KPIs, risk heatmap, bottleneck chart, what-if simulator
- **Leadership** tab — ROI projections, model validation, implementation roadmap

Use the **Step** / **Auto-Play** controls to advance the simulation through a 30-step shift.

Run the Standalone Demo

```
python demo.py
```

The demo runs in **~15 seconds** and outputs:
1. **Terminal output** — Step-by-step inference with formatted tables
2. **4 visualization PNGs** — Saved to `demo_outputs/`

📊 Dashboard Views

🔧 Floor Supervisor — Real-Time Operations

- **Live Assembly Graph** — 40 stations rendered as an interactive graph, color-coded by risk level (green/amber/red) with real-time prediction overlays
- **Active Alerts** — Prioritized alert feed showing stations with escalating risk, with root-cause context
- **Station Status Matrix** — Sortable table with risk, queue, anomaly score, and confidence for all 40 stations
- **Virtual Sensing Panel** — Highlights under-instrumented (PARTIAL/MANUAL) stations with GNN-inferred confidence scores

📊 Plant Manager — Shift Planning & Trends

- **Shift KPIs** — High/medium risk counts, average queue, estimated OEE, low-confidence stations, anomaly rate
- **Risk Heatmap** — Station × Time heatmap showing risk evolution across the shift
- **Bottleneck Frequency** — Ranked bar chart of most frequently flagged stations
- **What-If Simulator** — Select a station + intervention (maintenance, takt increase, reroute) and see predicted downstream impact
- **Anomaly Scenario Tracker** — Live status of active anomaly scenarios with progression

💼 Leadership — Business Case & ROI

- **ROI Projection Cards** — Downtime reduction, rework savings, bottleneck lead time, sensor cost savings

- **Model Validation Metrics** — Precision, recall, F1, false alarm rate computed live against ground truth
- **Multi-Plant Rollout Plan** — Scaling strategy from pilot to enterprise
- **Implementation Roadmap** — Phased timeline with investment milestones
- **Technical Architecture** — Data / Intelligence / Application layer breakdown

🎯 Key Features

1. Predictive Propagation

The model detects anomalous conditions (e.g., weld gun wear at Station 7, humidity excursion in Paint Booth) and predicts downstream cascade — **before** defects propagate through the line.

2. Virtual Sensing for Legacy Equipment

40% of stations run on legacy equipment with partial or manual-only instrumentation. The GNN infers missing sensor readings via masked-node training and neighbor aggregation, flagging low-confidence stations for manual inspection instead of trusting unreliable data.

3. What-If Simulation

Test interventions virtually: "What happens if we perform maintenance on Station 7?" — see predicted queue reduction and risk improvement across the entire downstream graph, enabling data-driven maintenance scheduling without disrupting production.

4. Multi-Stakeholder Dashboard

Three purpose-built views — floor supervisor (real-time, in-the-moment), plant manager (shift planning, trends), leadership (ROI, validation) — from the same underlying model, driving adoption at every organizational level.

5. Multi-Causal Anomaly Detection

Five realistic anomaly scenarios covering equipment wear, environmental excursions, operator variation, and upstream supply quality — with cascade propagation through the graph topology.

🔮 Scalability & Future Work

- **Scale to 100+ stations** with hierarchical graph pooling
- **Real-time streaming** via Apache Kafka + GNN inference server
- **Multi-plant federation** with graph-of-graphs architecture
- **Reinforcement learning** for automated intervention recommendations
- **Integration with MES/ERP** systems for closed-loop optimization

- **Edge deployment** with quantized GNN models on plant-floor hardware

📈 Business Impact

METRIC	EXPECTED IMPROVEMENT
Unplanned Downtime	30–45% reduction
Rework Costs	25–35% reduction
Bottleneck Detection Lead Time	From reactive to 10–15 min advance warning
Sensor Hardware Costs	40–60% reduction via virtual sensing
Annual Savings (per plant)	$2.8M–$4.5M
Pilot Payback Period	~6–9 months

🛠️ Technology Stack

- **Python 3.9+**
- **PyTorch 2.0+** — Deep learning framework
- **PyTorch Geometric** — Graph neural network library (GATv2Conv)
- **Flask** — Web application framework
- **NetworkX** — Graph construction and analysis
- **Matplotlib** — Static visualization (demo mode)
- **NumPy** — Numerical computing
- **JavaScript (Vanilla)** — Dashboard frontend

📜 License

MIT License — see LICENSE for details.

DigitalTwin.ai — *From reactive firefighting to proactive optimization.* *Built with ❤️ for the Accenture Innovation Challenge 2026*